

## 14.4 - Using Wildcard Types in Generics

### Introduction

In Java Generics, a **wildcard** represents an unknown type. This is useful when working with collections where the type parameter is not specified or can be any type. Generics with a specified type parameter provide type safety by ensuring that only a specific class type of data can be added to a collection. In contrast, wildcards enable greater flexibility while still retaining some level of type safety.

### Using Wildcards in Generics

Wildcards allow you to work with collections in a more flexible way. When defining a generic type, the diamond operator ( $\diamond$ ) specifies the type, ensuring type safety. However, if you use the question mark symbol (?), also known as a **wildcard**, you do not specify a concrete type.

- The ? (question mark) symbol represents a wildcard, indicating "any type" or "unknown type."
- Wildcards can be used in various places, such as:
  - Type parameters
  - Fields
  - Return types
  - Local variables

However, wildcards **cannot** be used:

- As a type argument for a generic method invocation
- For creating instances of a generic class
- As a supertype

## 👉 Example Code

Let's illustrate this concept with an example:

```
class Human {
    public void sleep() {
        System.out.println("Humans need to sleep well...");
    }
}

class Employee extends Human {
    @Override
    public void sleep() {
        System.out.println("Employees need to sleep well to stay productive...");
    }
}

public class Demo {
    public static void main(String[] args) {
        Human human = new Human();
        Employee emp = new Employee();

        // Example of runtime polymorphism:
        // A child reference (Employee) can be assigned to a parent class (Human) reference.
        human = emp;

        // Using collections with generics for type safety:
        ArrayList<Human> humanList = new ArrayList<>();
        ArrayList<Employee> empList = new ArrayList<>();

        // This assignment is not allowed:
        // humanList = empList;
        // In a collection of type Human, only Human type objects are allowed.
        // Assigning an Employee type collection is not permitted.

        // Assigning another Human collection is allowed:
        ArrayList<Human> humanList2 = new ArrayList<>();
        humanList = humanList2; // Valid since both collections are of type Human.

        // Modifying code to use wildcards:
        ArrayList<?> unknownList = new ArrayList<>();
        // Here, the reference type is replaced with a
```

```
wildcard (`?`), meaning the type is unknown.  
    // This allows flexibility, as any data type can fit  
    in the collection.  
  
    // Now, this assignment is valid:  
    unknownList = empList; // The wildcard makes it  
possible to assign empList.  
    }  
}
```

## Explanation

### 1. Runtime Polymorphism:

- In Java, a child class reference can be assigned to a parent class reference because they share an "is-a" relationship. For instance, an Employee can be treated as a Human.
- However, when using collections, you cannot assign a list of Employee objects to a list of Human objects because the type parameter in generics enforces strict type safety.

### 2. Using Wildcards:

- By replacing the type parameter with a wildcard (?), the collection can accept any type, making it more flexible.
- Wildcards are particularly useful when you want to pass collections to methods but do not know the exact type they should accept.

## Conclusion

Using wildcards in Java Generics provides flexibility for working with collections where the exact type may not be known at compile time. However, understanding the rules of assignment and bounded wildcards (both upper and lower bounds) is essential for proper usage.